

OpenSnippet CTF Write-up

Author: @elesquina

Challenge Overview

The challenge presented a code snippet sharing platform called SnipShare, accessible at <https://ideh-opensnippet.chals.io/>. The application allowed users to create and share code snippets without requiring authentication, featuring syntax highlighting for multiple programming languages and various expiration options.

Initial Reconnaissance

Initial exploration of the application revealed several key pages and features. The homepage displayed a clean interface promoting quick code snippet sharing. The navigation menu included options for creating new snippets, viewing statistics, and accessing an about page. The statistics page indicated the presence of 134 total snippets at the time of initial inspection and documented three API endpoints: POST `/api/snippets` for creating snippets, GET `/api/snippets/:id` for retrieving snippets, and GET `/api/stats` for viewing statistics.

Creating a test snippet revealed that the application used an eight-character identifier format. After submitting a snippet with the content "hello", the application redirected to `/s/-MlY5dzG`, indicating snippets used a nanoid-based identification scheme. The API endpoint `/api/snippets/-MlY5dzG` returned structured JSON data containing the snippet's metadata including ID, title, content, language, creation timestamp, expiration date, and view count.

Rate Limiting Discovery

Attempting to enumerate snippets by testing various IDs quickly encountered rate limiting restrictions. The application returned HTTP 429 responses with JSON error messages indicating "Too many requests" and providing retry-after timings. This rate limiting mechanism prevented brute-force enumeration of snippet IDs and protected the API from abuse.

The challenge provided a subtle hint with the phrase "try using host localhost". This suggested that the rate limiting mechanism might treat requests differently based on their apparent origin. Modern web applications deployed behind reverse proxies often use headers like X-Forwarded-For to determine the actual client IP address.

Bypassing Rate Limiting

Testing with the X-Forwarded-For header set to 127.0.0.1 successfully bypassed the rate limiting mechanism. Requests including this header were treated as originating from localhost and granted significantly higher rate limits or complete exemption from rate limiting. The following command executed without triggering rate limit errors:

```
curl https://ideh-opensnippet.chals.io/api/snippets/0 \
-H "X-Forwarded-For: 127.0.0.1"
```

This vulnerability arose from the application's trust proxy configuration. The Express.js application set `app.set('trust proxy', 1)`, instructing it to trust the first proxy in the chain and use the X-Forwarded-For header for determining client IP addresses. The rate limiting middleware then used this potentially spoofed IP address for tracking request counts.

Source Code Analysis

The challenge description mentioned the application was open source, leading to the discovery of the repository at <https://github.com/elEsquina/SnipShare>. Examining the source code revealed the complete application structure. The rate limiting middleware stored request counts in an in-memory Map using the client IP as the key, which came from `req.ip` that Express populated from the X-Forwarded-For header when trust proxy was enabled.

The database schema showed a simple snippets table with columns for ID, title, content, language, creation timestamp, expiration timestamp, and view count. The application used nanoid to generate eight-character random IDs for each snippet. No endpoint existed for listing all snippets, and the API only supported retrieval by exact ID match.

Git History Investigation

Reviewing the commit history proved crucial for solving the challenge. The commit log revealed a particularly interesting entry with the SHA `9458833bc52704765ff6e84c22c5ba2b178a90c3` and message: "Remove .env and add .gitignore - Oops! Accidentally committed my local .env file. Added proper .gitignore to prevent this in the future. # Comment by Dreekos: GG for finding my mistake! # Remember to NEVER push API keys or secrets to your repos!"

This commit indicated that the developer had accidentally committed environment configuration files containing sensitive information, then removed them in a subsequent commit. However, Git's version control system preserves all historical changes, making it possible to retrieve deleted files from previous commits.

Flag Retrieval

The commit immediately before the .env file deletion had SHA `59f4f5424f5d3f7dd3433513a29f1b53ebcfc603`. Accessing the .env file from this specific commit using the GitHub raw content URL revealed the configuration file:

```
curl https://raw.githubusercontent.com/elEsquina/SnipShare/\
59f4f5424f5d3f7dd3433513a29f1b53ebcfc603/.env
```

The .env file contained various configuration parameters including port settings, database paths, session secrets, API keys, and rate limiting parameters. Among these configuration values, the ADMIN_SECRET variable contained the flag:

```
IDEH{g1t_h1st0ry_d03s_n0t_f0rg3t}
```

Vulnerability Chain

The challenge demonstrated multiple security concepts working together. The rate limiting bypass using X-Forwarded-For header spoofing allowed unrestricted API access, though this alone was insufficient to solve the challenge. The real vulnerability lay in the exposure of sensitive information through Git history, a common mistake developers make when accidentally committing configuration files containing secrets.

The flag itself served as a reminder that Git history preserves all changes, and removing sensitive files in later commits does not truly delete them from the repository's history. This emphasizes the importance of never committing secrets to version control systems in the first place, as tools like git-secrets and pre-commit hooks should be employed to prevent such mistakes. Once secrets are committed, the entire repository history must be cleaned using tools like git-filter-branch or BFG Repo-Cleaner, or the secrets must be rotated immediately.

Key Takeaways

The OpenSnippet challenge effectively illustrated several critical security principles. First, the misconfiguration of trust proxy settings can lead to rate limiting bypasses when applications blindly trust client-provided headers. Second, Git repositories retain complete history of all commits, making accidentally committed secrets permanently accessible unless the history is rewritten. Third, security vulnerabilities often require chaining multiple weaknesses together, as the rate limiting bypass enabled exploration but the real flag was hidden in the Git history.

From a defensive perspective, developers should implement proper header validation when using trust proxy settings, ensure secrets are never committed to version control through automated checks, regularly audit Git history for sensitive information, and maintain separate configuration management systems for secrets rather than storing them in repository files. The

challenge successfully demonstrated how seemingly minor configuration mistakes can expose critical information to attackers with basic reconnaissance skills.